

Chapter 57

Remote Control Peripheral (RMT)

57.1 Overview

The Remote Control (RMT) module is designed to transmit and receive infrared remote control signals. A variety of remote control protocols can be encoded/decoded via software based on the RMT module. The RMT module converts pulse codes stored in the module's built-in RAM into output signals, or converts input signals into pulse codes and stores them in RAM. In addition, the RMT module optionally modulates its output signals with a carrier wave, or optionally demodulates and filters its input signals.

The RMT module has eight channels, numbered from zero to seven. Each channel is able to independently transmit or receive signals.

- Channel 0 ~ 3 (TX channel) are dedicated to transmitting signals;
- Channel 4 ~ 7 (RX channel) are dedicated to receiving signals.

Each TX/RX channel has the same functionality controlled by a dedicated set of registers and is able to independently transmit or receive data. Channel 3 and channel 7 support GDMA access, so the two channels also have a set of GDMA-related control and status registers. TX channels are indicated by *n* which is used as a placeholder for the channel number, and by *m* for RX channels.

57.2 Features

The RMT module has the following features:

- Eight channels:
 - TX channels 0 ~ 3
 - RX channels 4 ~ 7
 - Eight channels share a 384 x 32-bit RAM
- The transmitter supports:
 - Normal TX mode
 - Wrap TX mode
 - Continuous TX mode
 - Modulation on TX pulses
 - Multiple channels transmitting data simultaneously (programmable)
 - GDMA access supported by TX channel 3

- The receiver supports:
 - Normal RX mode
 - Wrap RX mode
 - RX filtering
 - Demodulation on RX pulses
 - GDMA access supported by RX channel 7

57.3 Functional Description

57.3.1 Architecture

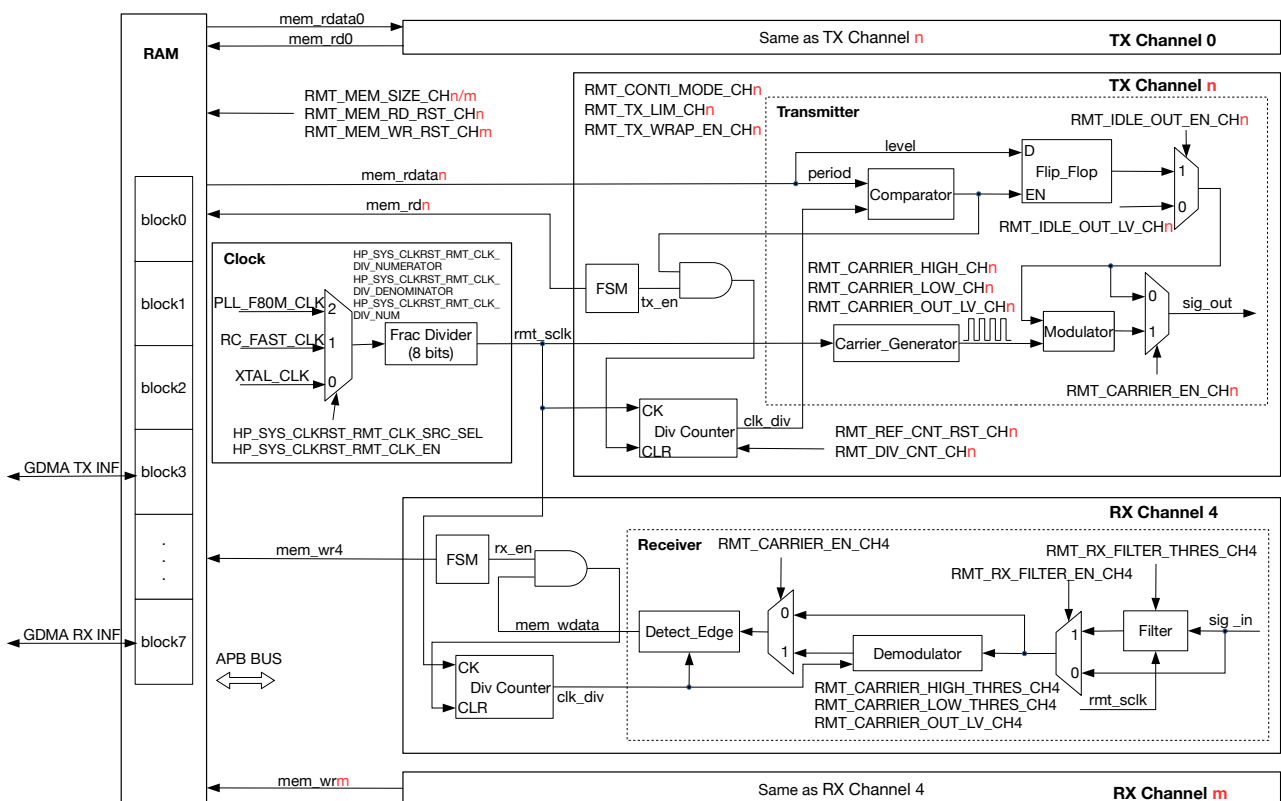


Figure 57.3-1. RMT Architecture

As shown in Figure 57.3-1, each TX channel has:

- 1 x clock divider counter (Div Counter)
- 1 x state machine (FSM)
- 1 x transmitter

Each RX channel has:

- 1 x clock divider counter (Div Counter)

- 1 x state machine (FSM)
- 1 x receiver

The eight channels share a 384 x 32-bit RAM.

57.3.2 RAM

57.3.2.1 Structure of RAM

Figure 57.3-2 shows the format of pulse code in RAM. Each pulse code contains a 16-bit entry with two fields: “level” and “period”. “level” (0 or 1) indicates a low-/high-level value that has been received or is going to be sent, while “period” points out the number of clock cycles (see `clk_div` in Figure 57.3-1) that the level lasts for.

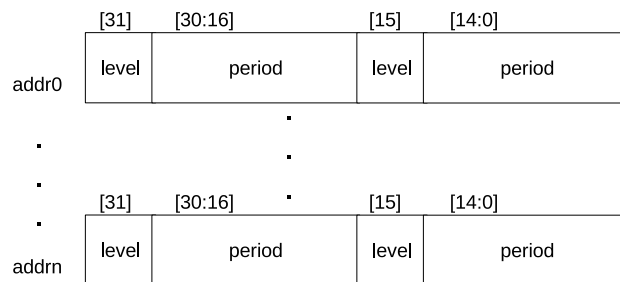


Figure 57.3-2. Format of Pulse Code in RAM

The minimum value for the period is zero (0) and is interpreted as a transmission end-marker. For a non-zero period (i.e., not an end-marker), its value is limited by APB clock and RMT clock according to the formula below:

$$5 \times T_{apb_clk} + 6 \times T_{rmt_sclk} < period \times T_{clk_div} \quad (57.1)$$

Note:

RMT clock and APB clock must satisfy the following relation:

$$1.5 \times T_{apb_clk} < 9 \times T_{rmt_sclk} \quad (57.2)$$

According to the formula above and the frequency of `rmt_sclk`, the pulse width (i.e., $period \times T_{clk_div}$) able to be captured by RMT is limited as follows:

- The minimum value of pulse width should be larger than $(5 \times T_{apb_clk} + 6 \times T_{rmt_sclk})$.
- The maximum value of pulse width should be smaller than or equal to (the maximum period $\times$ the maximum T_{clk_div}), i.e., $((2^{15} - 1) \times \text{the maximum } T_{rmt_sclk} \times 256)$.

For more information about the `rmt_sclk` frequency, or `APB_CLK` frequency, see Section 57.3.3, or Chapter 10 [Reset and Clock](#).

57.3.2.2 Use of RAM

The RAM is divided into eight 48 x 32-bit blocks. By default, each channel uses one block (block 0 for channel 0, block 1 for channel 1, and so on).

If the data size of one single transfer is larger than one block size of TX channel n or RX channel m , users can configure the channel:

- to enable [wrap mode](#) by setting [RMT_MEM_TX/RX_WRAP_EN_CH \$n/m\$](#) ;
- or to use more blocks by configuring [RMT_MEM_SIZE_CH \$n/m\$](#) .

Setting [RMT_MEM_SIZE_CH \$n/m\$](#) > 1 allows channel n/m to use the memory of the subsequent channels, i.e., block $(n/m) \sim$ block $(n/m + \text{RMT_MEM_SIZE_CH}_{n/m} - 1)$. In such case, the subsequent channels $(n/m + 1) \sim (n/m + \text{RMT_MEM_SIZE_CH}_{n/m} - 1)$ can not be used since their RAM blocks are occupied. For example, if channel 0 is configured to use block 0 and block 1, then channel 1 will be unavailable since its block is occupied, while channel 2 and channel 3 are not affected and can be used normally.

Note that the RAM used by each channel is mapped from low address to high address. Under such mapping, channel 0 is able to use the RAM blocks for channels 1, 2 ... and 7 by setting [RMT_MEM_SIZE_CHO](#), but channel 7 can not use the blocks for channels 0, 1, ... or 6. Therefore, the value of [RMT_MEM_SIZE_CH \$n\$](#) should not exceed $(8 - n)$ and the value of [RMT_MEM_SIZE_CH \$m\$](#) should not exceed $(8 - m)$.

The RMT RAM can be accessed via APB bus, or read by the transmitter and written by the receiver. To avoid any possible access conflict between the receiver writing RAM and the APB bus reading RAM, RMT can be configured to designate the RAM block's owner, be it the receiver or the APB bus, by configuring [RMT_MEM_OWNER_CH \$m\$](#) . If this ownership is violated, a flag signal [RMT_MEM_OWNER_ERR_CH \$m\$](#) will be generated.

57.3.2.3 RAM Access

APB bus is able to access RAM in FIFO mode and in NONFIFO (Direct Address) mode, depending on the configuration of [RMT_APB_FIFO_MASK](#):

- 0: Use FIFO mode;
- 1: Use NONFIFO mode.

Channels 3 and 7 also support GDMA access.

FIFO Mode

In FIFO mode, the APB reads data from or writes data to RAM via a fixed address stored in [RMT_CH \$n/m\$ DATA_REG](#).

NONFIFO Mode

In NONFIFO mode, the APB writes data to or reads data from a continuous address range.

- The write-starting address of TX channel n is: RMT base address + 0x800 + $n \times 48$. The access address for the second data and the following data are RMT base address + 0x800 + $n \times 48$ + 0x4, and so on, incremented by 0x4.
- The read-starting address of RX channel m is: RMT base address + 0x8C0 + $m \times 48$. The access address for the second data and the following data are RMT base address + 0x8C0 + $m \times 48$ + 0x4, and so on, incremented by 0x4.

GDMA Mode

Channel 3 also supports GDMA access. If [RMT_DMA_ACCESS_EN_CH3](#) is set, RAM of channel 3 only allows GDMA access. FIFO access or NOFIFO access to channel 3 by the APB bus are forbidden, otherwise **unpredictable consequences may occur**.

To ensure correct data transmission,

1. GDMA should be started first.
2. RMT can only start transmitting data after GDMA channel gets data ready, otherwise, unexpected data may be sent.

In [normal TX mode](#), when the RAM of channel 3 is fully written by GDMA, an [RMT_APB_MEM_WR_ERR_CH3](#) interrupt is triggered. Setting [RMT_MEM_TX_WRAP_EN_CH3](#) allows channel 3 to transmit more data than one block can fit, with no software operation needed.

Channel 7 also supports GDMA access. If [RMT_DMA_ACCESS_EN_CH7](#) is set, the RAM of channel 7 is allowed to transmit data to GDMA. Note in this mode, channel 7's RAM can also be accessed by APB via NONFIFO mode.

In [normal RX mode](#), when the size of data read by GDMA from channel 7 is equal to its RAM size, an [RMT_APB_MEM_RD_ERR_CH7](#) is triggered and the subsequent data is discarded. If [RMT_MEM_RX_WRAP_EN_CH7](#) is set, data of more than one block size can be received with no software wrap operation needed. If channel 7's RAM is full but the GDMA still does not start receiving data from the channel, the newly received data by this channel will replace the previous data.

Note:

When channel 7 receives an end-marker, a GDMA `in_suc_eof` interrupt is generated. Two bytes are written to GDMA if the `period[14:0]` is 0, and four bytes to GDMA if the `period[30:16]` is 0.

57.3.3 Clock

The clock source of RMT can be `PLL_F80M_CLK`, `RC_FAST_CLK`, or `XTAL_CLK`, depending on the configuration of [HP_SYS_CLKRST_RMT_CLK_SRC_SEL](#). RMT clock can be enabled by setting [HP_SYS_CLKRST_RMT_CLK_EN](#). RMT working clock (see `rmt_sclk` in Figure 57.3-1) is obtained by dividing the selected clock source with a fractional divider. The divider is $\text{HP_SYS_CLKRST_RMT_CLK_DIV_NUM} + 1 + \text{HP_SYS_CLKRST_RMT_CLK_DIV_NUMERATOR}$ or [HP_SYS_CLKRST_RMT_CLK_DIV_DENOMINATOR](#).

For more information, see Chapter 10 [Reset and Clock](#). [RMT_DIV_CNT_CHn/m](#) is used to configure the divider coefficient of internal clock divider for RMT channels. The coefficient is normally equal to the value of [RMT_DIV_CNT_CHn/m](#), except value 0 that represents divider 256. The clock divider can be reset by setting [RMT_REF_CNT_RST_CHn/m](#). The clock generated from the divider can be used by the counter (see Figure 57.3-1).

57.3.4 Transmitter

Note:

Updating the configuration described in this and subsequent sections requires setting `RMT_CONF_UPDATE_CH $n$ / $m$` first. See Section 57.3.6.

57.3.4.1 Normal TX Mode

When `RMT_TX_START_CH $n$` is set, the transmitter of channel n starts reading and transmitting pulse codes from the starting address of its RAM block. The codes are sent starting from low-address entry. When an end-marker (a zero period) is encountered, the transmitter stops the transmission, returns to idle state and generates an `RMT_CH $n$ _TX_END_INT` interrupt. Setting `RMT_TX_STOP_CH $n$` to 1 also stops the transmission and immediately sets the transmitter back to idle. The output level of a transmitter in idle state is determined by the “level” field of the end-marker or by the content of `RMT_IDLE_OUT_LV_CH $n$` , depending on the configuration of `RMT_IDLE_OUT_EN_CH $n$` :

- 0: The level in idle state is determined by the “level” field of the end-marker;
- 1: The level is determined by `RMT_IDLE_OUT_LV_CH $n$` .

57.3.4.2 Wrap TX Mode

To transmit more pulse codes than that can be fitted in the channel's RAM, users can enable wrap TX mode for channel n by setting `RMT_MEM_TX_WRAP_EN_CH $n$` . In this mode, the transmitter transmits the data from RAM in loops till an end-marker is encountered. For example, if `RMT_MEM_SIZE_CH $n$` = 1, the transmitter starts transmitting data from the address $48 \times n$, and then the data from higher RAM address. Once the transmitter finishes transmitting data from $(48 \times (n + 1) - 1)$, it continues transmitting data from $48 \times n$ again till an end-marker is encountered. Wrap mode is also applicable for `RMT_MEM_SIZE_CH $n$` > 1.

When the size of transmitted pulse codes is larger than or equal to the value set by `RMT_TX_LIM_CH $n$` , an `RMT_CH $n$ _TX_THR_EVENT_INT` interrupt is generated. In wrap mode, `RMT_TX_LIM_CH $n$` can be set to a half or a fraction of the size of the channel's RAM block. When an `RMT_CH $n$ _TX_THR_EVENT_INT` interrupt is detected by software, the already used RAM region can be updated by new pulse codes. In such way, the transmitter can seamlessly transmit unlimited pulse codes in wrap mode.

Note:

If RAM is accessed by GDMA, more pulse codes than one block size can be transmitted with no additional operation needed. If accessed by APB bus, wrap mode has to be enabled via software to transmit more data than one block size.

57.3.4.3 TX Modulation

Transmitter output can be modulated with a carrier wave by setting `RMT_CARRIER_EN_CH $n$` . The carrier waveform is configurable. In a carrier cycle, the high level lasts for $(\text{RMT_CARRIER_HIGH_CH}_n + 1) \text{ rmt_sclk}$ cycles, while the low level lasts for $(\text{RMT_CARRIER_LOW_CH}_n + 1) \text{ rmt_sclk}$ cycles. When `RMT_CARRIER_OUT_LV_CH $n$` is set, carrier wave is added on the high-level of output signals; while `RMT_CARRIER_OUT_LV_CH $n$` is cleared, carrier wave is added on the low-level of output signals. Carrier wave can be added on all output signals during modulation, or just added on valid pulse codes (the data stored in RAM), which can be set by configuring `RMT_CARRIER_EFF_EN_CH $n$` :

- 0: Add carrier wave on all output signals;

- 1: Add carrier wave only on valid signals.

57.3.4.4 Continuous TX Mode

The continuous TX mode can be enabled by setting `RMT_TX_CONTI_MODE_CHn`. In this mode, the transmitter transmits the pulse codes from RAM in loops:

- If an end-marker is encountered, the transmitter starts transmitting from the first data of the channel's RAM again.
- If no end-marker is encountered, the transmitter starts transmitting from the first data again after the last data is transmitted.

If `RMT_TX_LOOP_CNT_EN_CHn` is set, the loop counting is incremented by 1 each time an end-marker is encountered. If the counting reaches the value set by `RMT_TX_LOOP_NUM_CHn`, an `RMT_CHn_TX_LOOP_INT` interrupt is generated. If `RMT_LOOP_STOP_EN_CHn` is set, the transmission stops immediately once an `RMT_CHn_TX_LOOP_INT` interrupt is generated, otherwise, the transmission will continue. In an end-maker, if its period[14:0] is 0, then the period of the previous data must satisfy:

$$10 \times T_{apb_clk} + 19 \times T_{rmt_sclk} < period \times T_{clk_div} \quad (57.3)$$

The period of the other data only need to satisfy [non-zero period relation](#).

57.3.4.5 Simultaneous TX Mode

RMT module supports multiple channels transmitting data simultaneously. To use this function, follow the steps below:

1. Configure `RMT_TX_SIM_CHn` to choose which multiple channels are used to transmit data simultaneously.
2. Set `RMT_TX_SIM_EN` to enable this transmission mode.
3. Set `RMT_TX_START_CHn` for each selected channel, to start data transmitting.

The transmission starts once the final channel is configured. RMT module also supports simultaneous transmission of channels 0 ~ 2's RAM accessed by APB bus and channel 3's RAM accessed by GDMA.

57.3.5 Receiver

57.3.5.1 Normal RX Mode

The receiver of channel *m* is controlled by `RMT_RX_EN_CHm`:

- 0: The receiver stops receiving data;
- 1: The receiver starts working.

When the receiver becomes active, it starts counting from the first edge of the signal, detecting signal levels and counting clock cycles the level lasts for. Each cycle count (period) is then written back to RAM together with the level information (level). When the receiver detects no change in a signal level for a number of clock cycles more than the value set by `RMT_IDLE_THRES_CHm`, the receiver will stop receiving data, return to idle state, and generate an `RMT_CHm_RX_END_INT` interrupt. Please note that `RMT_IDLE_THRES_CHm` should be

configured to a maximum value according to your application, otherwise a valid received level may be mistaken as a level in idle state. If the RAM space of this RX channel is used up by the received data, the receiver stops receiving data, and an `RMT_CH $m$ _ERR_INT` interrupt is triggered by RAM FULL event.

57.3.5.2 Wrap RX Mode

To receive more pulse codes than can be fitted in the channel's RAM, users can enable wrap mode for channel m by configuring `RMT_MEM_RX_WRAP_EN_CH $m$` . But if RAM is accessed by GDMA, more pulse codes than one block size can be received with no additional operation needed. If accessed by APB bus, wrap mode has to be enabled to transmit more data than one block size. In wrap mode, the receiver stores the received data to RAM space of this channel in loops. Receiving ends, when the receiver detects no change in a signal level for a number of clock cycles more than the value set by `RMT_IDLE_THRES_CH $m$` . The receiver returns to idle state and generates an `RMT_CH $m$ _RX_END_INT` interrupt. For example, if `RMT_MEM_SIZE_CH $m$` is set to 1, the receiver starts receiving data and stores the data to address $48 * m$, and then to higher RAM address. When the receiver finishes storing the received data to $(48 * (m + 1) - 1)$, the receiver continues receiving data and storing data to the address $48 * m$ again, till no change is detected on a signal level for more than `RMT_IDLE_THRES_CH $m$` clock cycles. Wrap mode is also applicable for `RMT_MEM_SIZE_CH $m$` > 1.

An `RMT_CH $m$ _RX_THR_EVENT_INT` interrupt is generated when the size of received pulse codes is larger than or equal to the value set by `RMT_CH $m$ _RX_LIM_REG`. In wrap mode, `RMT_CH $m$ _RX_LIM_REG` can be set to a half or a fraction of the size of the channel's RAM block. When an `RMT_CH $m$ _RX_THR_EVENT_INT` interrupt is detected, the already used RAM region can be updated by subsequent data.

57.3.5.3 RX Filtering

Users can enable the receiver to filter input signals by setting `RMT_RX_FILTER_EN_CH $m$` for channel m . The filter samples input signals continuously, and detects the signals which remain unchanged for a continuous `RMT_RX_FILTER_THRES_CH $m$` rmt_sclk cycles as valid, otherwise, the signals will be detected as invalid. Only the valid signals can pass through this filter. The filter removes pulses with a length of less than `RMT_RX_FILTER_THRES_CH $m$` rmt_sclk cycles.

57.3.5.4 RX Demodulation

Users can enable RX demodulation on input signals or on filtered signals by setting `RMT_CARRIER_EN_CH $m$` . RX demodulation can be applied to high-level carrier wave or low-level carrier wave, depending on the configuration of `RMT_CARRIER_OUT_LV_CH $m$` :

- 0: Demodulate low-level carrier wave;
- 1: Demodulate high-level carrier wave.

Users can configure `RMT_CARRIER_HIGH_THRES_CH $m$` and `RMT_CARRIER_LOW_THRES_CH $m$` to set the thresholds to demodulate high-level carrier or low-level carrier. If the high-level of a signal lasts for less than `RMT_CARRIER_HIGH_THRES_CH $m$` clk_div cycles, or the low-level lasts for less than `RMT_CARRIER_LOW_THRES_CH $m$` clk_div cycles, such level is detected as a carrier and then is filtered out.

57.3.6 Configuration Update

To update RMT registers configuration, please set `RMT_CONF_UPDATE_CH $n/m$` for each channel first. All the bits/fields listed in the second column of Table 57.3-1 should follow this rule.

Table 57.3-1. Configuration Update

Register	Bit/Field Configuration Update
TX Channel	
RMT_CH n CONFO_REG	RMT_CARRIER_OUT_LV_CH n
	RMT_CARRIER_EN_CH n
	RMT_CARRIER_EFF_EN_CH n
	RMT_DIV_CNT_CH n
	RMT_TX_STOP_CH n
	RMT_IDLE_OUT_EN_CH n
	RMT_IDLE_OUT_LV_CH n
	RMT_TX_CONTI_MODE_CH n
RMT_CH n CARRIER_DUTY_REG	RMT_CARRIER_HIGH_CH n
	RMT_CARRIER_LOW_CH n
RMT_CH n TX_LIM_REG	RMT_TX_LOOP_CNT_EN_CH n
	RMT_TX_LOOP_NUM_CH n
	RMT_TX_LIM_CH n
RMT_TX_SIM_REG	RMT_TX_SIM_EN
RX Channel	
RMT_CH m CONFO_REG	RMT_CARRIER_OUT_LV_CH m
	RMT_CARRIER_EN_CH m
	RMT_IDLE_THRES_CH m
	RMT_DIV_CNT_CH m
RMT_CH m CONF1_REG	RMT_RX_FILTER_THRES_CH m
	RMT_RX_EN_CH m
RMT_CH m RX_CARRIER_RM_REG	RMT_CARRIER_HIGH_THRES_CH m
	RMT_CARRIER_LOW_THRES_CH m
RMT_CH m RX_LIM_REG	RMT_RX_LIM_CH m
RMT_REF_CNT_RST_REG	RMT_REF_CNT_RST_CH m

57.4 Interrupts

ESP32-P4's RMT can generate the RMT_INTR interrupt signal that will be sent to the [Interrupt Matrix](#).

The following interrupt sources can generate the RMT_INTR interrupt signal:

- RMT_CH n _ERR_INT: Triggered when channel n does not read or write data correctly. For example, the transmitter still tries to read data from RAM when the RAM is empty.
- RMT_CH m _ERR_INT: Triggered when channel m does not read or write data correctly. For example, the receiver still tries to write data into RAM when the RAM is full.
- RMT_CH n _TX_THR_EVENT_INT: Triggered when the amount of data the transmitter has sent matches the value of RMT_CH n _TX_LIM_REG.
- RMT_CH m _RX_THR_EVENT_INT: Triggered each time when the amount of data received by the receiver reaches the value set in RMT_CH m _RX_LIM_REG.
- RMT_CH n _TX_END_INT: Triggered when the transmitter has finished transmitting signals.

- RMT_CH m _RX_END_INT: Triggered when the receiver has finished receiving signals.
- RMT_CH n _TX_LOOP_INT: Triggered when the loop counting reaches the value set by [RMT_TX_LOOP_NUM_CH \$n\$](#) in continuous TX mode.
- RMT_CH3_DMA_ACCESS_FAIL_INT: Triggered when the result of (the entries written to channel 3's RAM - the entries transmitted by channel 3) is larger than channel 3's RAM size, but DAM keeps writing data to this channel.
- RMT_CH7_DMA_ACCESS_FAIL_INT: Triggered when the result of (the entries received by channel 7's RAM - the entries read by GDMA) is larger than channel 7's RAM size, but channel 7 keeps receiving data.

Note:

For definitions of *interrupt*, *interrupt signal*, *interrupt source*, and their correlations, please refer to Chapter [12 Interrupt Matrix](#) > Section [12.2 Interrupt Terminology in ESP32-P4](#).

Each interrupt source can be configured by a common set of registers that are described in Section [Interrupt Configuration Registers](#). The specific registers can be found in Section [57.5 Register Summary](#).

57.5 Register Summary

The addresses in this section are relative to Remote Control Peripheral base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
FIFO R/W registers			
RMT_CH0DATA_REG	The read and write data register for channel 0 by APB FIFO access	0x0000	HRO
RMT_CH1DATA_REG	The read and write data register for channel 1 by APB FIFO access	0x0004	HRO
RMT_CH2DATA_REG	The read and write data register for channel 2 by APB FIFO access	0x0008	HRO
RMT_CH3DATA_REG	The read and write data register for channel 3 by APB FIFO access	0x000C	HRO
RMT_CH4DATA_REG	The read and write data register for channel 4 by APB FIFO access	0x0010	HRO
RMT_CH5DATA_REG	The read and write data register for channel 5 by APB FIFO access	0x0014	HRO
RMT_CH6DATA_REG	The read and write data register for channel 6 by APB FIFO access	0x0018	HRO
RMT_CH7DATA_REG	The read and write data register for channel 7 by APB FIFO access	0x001C	HRO
Configuration registers			
RMT_CH0CONFO_REG	Configuration register 0 for channel 0	0x0020	varies
RMT_CH1CONFO_REG	Configuration register 0 for channel 1	0x0024	varies
RMT_CH2CONFO_REG	Configuration register 0 for channel 2	0x0028	varies
RMT_CH3CONFO_REG	Configuration register 0 for channel 3	0x002C	varies
RMT_CH4CONFO_REG	Configuration register 0 for channel 4	0x0030	R/W
RMT_CH4CONF1_REG	Configuration register 1 for channel 4	0x0034	varies
RMT_CH5CONFO_REG	Configuration register 0 for channel 5	0x0038	R/W
RMT_CH5CONF1_REG	Configuration register 1 for channel 5	0x003C	varies
RMT_CH6CONFO_REG	Configuration register 0 for channel 6	0x0040	R/W
RMT_CH6CONF1_REG	Configuration register 1 for channel 6	0x0044	varies
RMT_CH7CONFO_REG	Configuration register 0 for channel 7	0x0048	R/W
RMT_CH7CONF1_REG	Configuration register 1 for channel 7	0x004C	varies
RMT_CH4_RX_CARRIER_RM_REG	Carrier remove register for channel 4	0x0090	R/W
RMT_CH5_RX_CARRIER_RM_REG	Carrier remove register for channel 5	0x0094	R/W
RMT_CH6_RX_CARRIER_RM_REG	Carrier remove register for channel 6	0x0098	R/W
RMT_CH7_RX_CARRIER_RM_REG	Carrier remove register for channel 7	0x009C	R/W
RMT_SYS_CONF_REG	RMT APB configuration register	0x00C0	R/W
RMT_REF_CNT_RST_REG	RMT clock divider reset register	0x00C8	WT
Status registers			

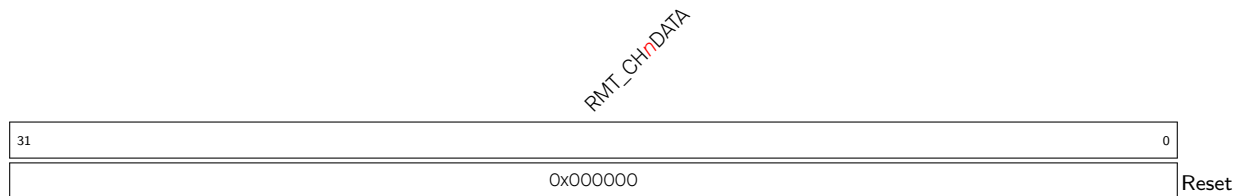
Name	Description	Address	Access
RMT_CH0STATUS_REG	Status register for channel 0	0x0050	RO
RMT_CH1STATUS_REG	Status register for channel 1	0x0054	RO
RMT_CH2STATUS_REG	Status register for channel 2	0x0058	RO
RMT_CH3STATUS_REG	Status register for channel 3	0x005C	RO
RMT_CH4STATUS_REG	Status register for channel 4	0x0060	RO
RMT_CH5STATUS_REG	Status register for channel 5	0x0064	RO
RMT_CH6STATUS_REG	Status register for channel 6	0x0068	RO
RMT_CH7STATUS_REG	Status register for channel 7	0x006C	RO
Interrupt registers			
RMT_INT_RAW_REG	Interrupt raw signal status register	0x0070	R/WTC/SS
RMT_INT_ST_REG	Interrupt signal status register	0x0074	RO
RMT_INT_ENA_REG	Interrupt enable signal configuration register	0x0078	R/W
RMT_INT_CLR_REG	Interrupt clear signal configuration register	0x007C	WT
Carrier wave duty cycle registers			
RMT_CH0CARRIER_DUTY_REG	Duty cycle configuration register for channel 0	0x0080	R/W
RMT_CH1CARRIER_DUTY_REG	Duty cycle configuration register for channel 1	0x0084	R/W
RMT_CH2CARRIER_DUTY_REG	Duty cycle configuration register for channel 2	0x0088	R/W
RMT_CH3CARRIER_DUTY_REG	Duty cycle configuration register for channel 3	0x008C	R/W
TX event configuration registers			
RMT_CH0_TX_LIM_REG	Configuration register for channel 0 TX event	0x00A0	varies
RMT_CH1_TX_LIM_REG	Configuration register for channel 1 TX event	0x00A4	varies
RMT_CH2_TX_LIM_REG	Configuration register for channel 2 TX event	0x00A8	varies
RMT_CH3_TX_LIM_REG	Configuration register for channel 3 TX event	0x00AC	varies
RMT_TX_SIM_REG	RMT TX synchronous register	0x00C4	R/W
RX event configuration registers			
RMT_CH4_RX_LIM_REG	Configuration register for channel 4 RX event	0x00B0	R/W
RMT_CH5_RX_LIM_REG	Configuration register for channel 5 RX event	0x00B4	R/W
RMT_CH6_RX_LIM_REG	Configuration register for channel 6 RX event	0x00B8	R/W
RMT_CH7_RX_LIM_REG	Configuration register for channel 7 RX event	0x00BC	R/W
Version register			
RMT_DATE_REG	Version control register	0x00CC	R/W

57.6 Registers

The addresses in this section are relative to Remote Control Peripheral base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

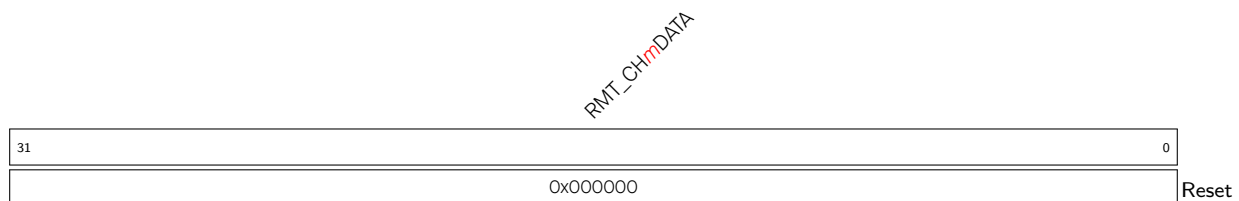
For how to program reserved fields, please refer to Section *Programming Reserved Register Field*.

Register 57.1. RMT_CH n DATA_REG (n : 0-3) (0x0000+0x4* n)



RMT_CH n DATA Read and write data for channel n via APB FIFO. (HRO)

Register 57.2. RMT_CH m DATA_REG (m : 4-7) (0x0000+0x4* m)



RMT_CH m DATA Read and write data for channel n via APB FIFO. (HRO)

Register 57.3. RMT_CH n CONF0_REG (n : 0-3) (0x0020+0x4* n)

(reserved)																			RMT_DMA_ACCESS_EN_CH3																			RMT_CONF_UPDATE_CHn																			(reserved)																			RMT_CARRIER_OUT_LV_CHn																			RMT_CARRIER_EN_CHn																			RMT_CARRIER_EFF_EN_CHn																			RMT_MEM_SIZE_CHn																																						RMT_DIV_CNT_CHn																																						RMT_TX_STOP_CHn																			RMT_IDLE_OUT_EN_CHn																			RMT_IDLE_OUT_LV_CHn																			RMT_MEM_TX_WRAP_EN_CHn																			RMT_TX_CONTI_MODE_CHn																			RMT_APB_MEM_RST_C																			RMT_MEM_RD_RST_C																			RMT_TX_STA																		
31								26								25		24		23		22		21		20		19								16								15								8								7		6		5		4		3		2		1		0																																																																																																																																																																																																																																																																																														
0								0								0		0		0		1		1		1		0x1								0x2								0								0		0		0		0		0		0		0		0		Reset																																																																																																																																																																																																																																																																																																				

RMT_TX_START_CH n Configures whether to enable sending data in channel n .

0: No effect

1: Enable

(WT)

RMT_MEM_RD_RST_CH n Configures whether to reset RAM read address accessed by the transmitter for channel n .

0: No effect

1: Reset

(WT)

RMT_APB_MEM_RST_CH n Configures whether to reset RAM W/R address accessed by APB FIFO for channel n .

0: No effect

1: Reset

(WT)

RMT_TX_CONTI_MODE_CH n Configures whether to enable continuous TX mode for channel n .

0: No Effect

1: Enable

In this mode, the transmitter starts transmission from the first data. If an end-marker is encountered, the transmitter starts transmitting data from the first data again; if no end-marker is encountered, the transmitter starts transmitting the first data again when the last data is transmitted.

(R/W)

RMT_MEM_TX_WRAP_EN_CH n Configures whether to enable wrap TX mode for channel n .

0: No effect

1: Enable

In this mode, if the TX data size is larger than the channel's RAM block size, the transmitter continues transmitting the first data to the last data in loops.

(R/W)

RMT_IDLE_OUT_LV_CH n Configures the level of output signal for channel n when the transmitter is in idle state. (R/W)

Continued on the next page...

Register 57.3. RMT_CH n CONFO_REG (n : 0-3) (0x0020+0x4* n)

Continued from the previous page...

RMT_IDLE_OUT_EN_CH n Configures whether to enable the output for channel n in idle state.

0: No effect

1: Enable

(R/W)

RMT_TX_STOP_CH n Configures whether to stop the transmitter of channel n sending data out.

0: No effect

1: Stop

(R/W/SC)

RMT_DIV_CNT_CH n Configures the divider for clock of channel n .

Measurement unit: rmt_sclk

(R/W)

RMT_MEM_SIZE_CH n Configures the maximum number of memory blocks allocated to channel n .

(R/W)

RMT_CARRIER_EFF_EN_CH n Configures whether to add carrier modulation on the output signal only at data-sending state for channel n .

0: Add carrier modulation on the output signal at data-sending state and idle state for channel n

1: Add carrier modulation on the output signal only at data-sending state for channel n

Only valid when RMT_CARRIER_EN_CH n is 1.

(R/W)

RMT_CARRIER_EN_CH n Configures whether to enable the carrier modulation on output signal for channel n .

0: Disable

1: Enable

(R/W)

RMT_CARRIER_OUT_LV_CH n Configures the position of carrier wave for channel n .

0: Add carrier wave on low level

1: Add carrier wave on high level

(R/W)

RMT_CONF_UPDATE_CH n Synchronization bit for channel n . (WT)

RMT_DMA_ACCESS_EN_CH3 Configures whether to enable the DMA access function for channel

3. This field is reserved for channel 0 ~ 2.

0: Disable

1: Enable

(R/W)

Register 57.4. RMT_CH m CONFO_REG (m : 4-7) (0x0030, 0x0038, 0x0040, 0x0048)

(reserved)				RMT_CARRIER_OUT_LV_CH ^m				RMT_CARRIER_EN_CH ^m				RMT_MEM_SIZE_CH ^m				RMT_DMA_ACCESS_EN_CH7				RMT_IDLE_THRES_CH ^m								RMT_DIV_CNT_CH ^m			
31	30	29	28	27	24	23	22									8	7									0					
0	0	1	1	0x1		0	0x7fff								0x2								Reset								

Reset

RMT_DIV_CNT_CH m Configures the clock divider of channel m .

Measurement unit: rmt_sclk

(R/W)

RMT_IDLE_THRES_CH m Configures RX threshold. When no edge is detected on the input signal for continuous clock cycles longer than this field value, the receiver stops receiving data.

Measurement unit: clk_div

(R/W)

RMT_DMA_ACCESS_EN_CH7 Configures whether to enable the DMA access function for channel 7. This field is reserved for channel 4 ~ 6.

0: Disable

1: Enable

(R/W)

RMT_MEM_SIZE_CH m Configures the maximum number of memory blocks allocated to channel m .

(R/W)

RMT_CARRIER_EN_CH m Configures whether to enable carrier demodulation on output signal for channel m .

0: Disable

1: Enable

(R/W)

RMT_CARRIER_OUT_LV_CH m Configures the position of carrier wave for channel m .

0: Demodulate carrier wave on low level

1: Demodulate carrier wave on high level

(R/W)

Register 57.5. RMT_CH m CONF1_REG (m : 4-7) (0x0034, 0x003C, 0x0044, 0x004C)

(reserved)																RMT_CONF_UPDATE_CH ^m			RMT_MEM_RX_WRAP_EN_CH ^m			RMT_RX_FILTER_THRES_CH ^m			RMT_RX_FILTER_EN_CH ^m			RMT_MEM_OWNER_CH ^m			RMT_APB_MEM_RST_CH ^m			RMT_MEM_WVR_RST_CH ^m			RMT_RX_EN_CH ^m			
31																16	15	14	13	12						5	4	3	2	1	0	Reset								
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0	0	0	0xf					0	1	0	0	0												

RMT_RX_EN_CH m Configures whether to enable the receiver to start receiving data in channel m .

0: Disable

1: Enable

(R/W)

RMT_MEM_WR_RST_CH*m* Configures whether to reset RAM write address accessed by the receiver for channel *m*.

0: No effect

1: Reset

(WT)

RMT_APB_MEM_RST_CH*m* Configures whether to reset RAM W/R address accessed by APB FIFO for channel *m*.

0: No effect

1: Reset

(WT)

RMT_MEM_OWNER_CH*m* Configures the ownership of channel *m*'s RAM block.

0: APB bus is using the RAM

1: Receiver is using the RAM

(R/W/SC)

RMT_RX_FILTER_EN_CH*m* Configures whether to enable the receiver's filter for channel *m*.

0: Disable

1: Enable

(R/W)

RMT_RX_FILTER_THRES_CH m Configures whether the receiver, when receiving data, ignores the input pulse when its width is shorter than this register value in units of rmt_sclk cycles.

0: No effect

1: Reset

(R/W)

RMT_MEM_RX_WRAP_EN_CH*m* Configures whether to enable wrap RX mode for channel *m*.

0: Disable

1: Enable

In this mode, if the RX data size is larger than channel *m*'s RAM block size, the receiver stores the RX data from the first address to the last address in loops.

(R/W)

RMT_CONF_UPDATE_CH m Synchronization bit for channel m . (WT)

Register 576. RMT_CH m _RX_CARRIER_RM_REG (m : 4-7) (0x0090, 0x0094, 0x0098, 0x009C)

Diagram illustrating the RMT_CARRIER register structure. The register is 32 bits wide, divided into two 16-bit fields. The left field is labeled `RMT_CARRIER_HIGH_THRES_CHm` and contains the value `0x00`. The right field is labeled `RMT_CARRIER_LOW_THRES_CHm` and also contains the value `0x00`. A `Reset` label is present at the bottom right.

RMT_CARRIER_LOW_THRES_CH m Configures the low level period in a carrier modulation mode for channel m , which is equal to (RMT_CARRIER_LOW_THRES_CH m + 1). (R/W)

RMT_CARRIER_HIGH_THRES_CH m Configures the high level period in a carrier modulation mode for channel m , which is equal to (RMT_CARRIER_HIGH_THRES_CH m + 1). (R/W)

Register 57.7. RMT_SYS_CONF_REG (0x00C0)

31	30	27	26	25	24	23	18	17	12	11	4	3	2	1	0
0	0	0	0	0	1	0x1	0x0	0x0	0x1	0x1	0	0	0	0	Reset

RMT_APB_FIFO_MASK Configures the memory access mode.

0: Access memory by FIFO

1: Access memory directly

(R/W)

RMT_CLK_EN Configures whether to enable signal of RMT register clock gate.

0: Power down the drive clock of registers

1: Power up the drive clock of registers

(R/W)

Register 57.8. RMT_REF_CNT_RST_REG (0x00C8)

(reserved)																								RMT_REF_CNT_RST_CH7 RMT_REF_CNT_RST_CH6 RMT_REF_CNT_RST_CH5 RMT_REF_CNT_RST_CH4 RMT_REF_CNT_RST_CH3 RMT_REF_CNT_RST_CH2 RMT_REF_CNT_RST_CH1 RMT_REF_CNT_RST_CH0									
31																								8	7	6	5	4	3	2	1	0	Reset
0 0																								0	0	0	0	0	0	0	0	0	

RMT_REF_CNT_RST_CH*n* (*n*: 0-3) Configures whether to reset the clock divider of channel *n*.
0: No effect
1: Reset
(WT)

RMT_REF_CNT_RST_CH*m* (*m*: 4-7) Configures whether to reset the clock divider of channel *m*.
0: No effect
1: Reset
(WT)

Register 57.9. RMT_CH n STATUS_REG (n : 0-3) (0x0050+0x4* n)

(reserved)					RMT_APB_MEM_WR_ERR_CH ⁿ		RMT_MEM_EMPTY_CH ⁿ		RMT_STATE_CH ⁿ		(reserved)					RMT_APB_MEM_WADDR_CH ⁿ		(reserved)		RMT_MEM_RADDR_EX_CH ⁿ						
31					27		26	25	24	22		21	20					11		10	9	0				
0					0		0	0	0	0		0	0					0		0	0					Reset

RMT_MEM_RADDR_EX_CH*n* Represents the memory address offset when transmitter of channel *n* is using the RAM. (RO)

RMT_APB_MEM_WADDR_CH*n* Represents the memory address offset when writes RAM over APB bus. (RO)

RMT_STATE_CH n Represents the FSM status of channel n . (RO)

RMT_MEM_EMPTY_CHn Represents whether the TX data size exceeds the memory size and the wrap TX mode is disabled.

0: Not exceed

1: Exceed

(RO)

RMT_APB_MEM_WR_ERR_CH n Represents whether the offset address exceeds memory size (overflows) when writes via APB bus.

0: Not exceed

1: Exceed

(RO)

Register 5710. RMT_CH m STATUS_REG (m : 4-7) (0x0050+0x4* m)

(reserved)				RMT_APB_MEM_RD_ERR_CH ^m				RMT_MEM_FULL_CH ^m				RMT_MEM_OWNER_ERR_CH ^m				RMT_STATE_CH ^m				(reserved)				RMT_APB_MEM_RADDR_CH ^m				(reserved)				RMT_MEM_WADDR_EX_CH ^m			
31	28	27	26	25	24	22	21	20					11	10	9																	0			
0	0	0	0	0	0	0	0	0	0xc0				0	0	0	0xc0																Reset			

RMT_MEM_WADDR_EX_CH m Represents the memory address offset when receiver of channel m is using the RAM. (RO)

RMT_APB_MEM_RADDR_CH m Represents the memory address offset when reads RAM over APB bus. (RO)

RMT_STATE_CH m Represents the FSM status of channel m . (RO)

RMT_MEM_OWNER_ERR_CH m Represents whether the ownership of memory block is wrong.
 0: The ownership of memory block is correct
 1: The ownership of memory block is wrong
 (RO)

RMT_MEM_FULL_CH m Represents whether the receiver receives more data than the memory can fit.
 0: The receiver does not receive more data than the memory can fit
 1: The receiver receives more data than the memory can fit
 (RO)

RMT_APB_MEM_RD_ERR_CH m Represents whether the offset address exceeds memory size (overflows) when reads RAM via APB bus.
 0: Not exceed
 1: Exceed
 (RO)

Register 57.11. RMT_INT_RAW_REG (0x0070)

(reserved) RMT_CH7_DMA_ACCESS_FAIL_INT_RAW RMT_CH3_DMA_ACCESS_FAIL_INT_RAW RMT_CH7_RX_THR_EVENT_INT_RAW RMT_CH6_RX_THR_EVENT_INT_RAW RMT_CH5_RX_THR_EVENT_INT_RAW RMT_CH4_RX_THR_EVENT_INT_RAW RMT_CH7_ERR_INT_RAW RMT_CH6_ERR_INT_RAW RMT_CH5_ERR_INT_RAW RMT_CH4_ERR_INT_RAW RMT_CH7_RX_END_INT_RAW RMT_CH6_RX_END_INT_RAW RMT_CH5_RX_END_INT_RAW RMT_CH4_RX_END_INT_RAW RMT_CH3_TX_LOOP_INT_RAW RMT_CH2_TX_LOOP_INT_RAW RMT_CH1_TX_LOOP_INT_RAW RMT_CH0_TX_THR_EVENT_INT_RAW RMT_CH3_TX_THR_EVENT_INT_RAW RMT_CH2_TX_THR_EVENT_INT_RAW RMT_CH1_TX_THR_EVENT_INT_RAW RMT_CH0_TX_THR_EVENT_INT_RAW RMT_CH3_TX_END_INT_RAW RMT_CH2_TX_END_INT_RAW RMT_CH1_TX_END_INT_RAW RMT_CH0_TX_END_INT_RAW																																
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

RMT_CH n _TX_END_INT_RAW (n : 0-3) The raw interrupt status of [RMT_CH \$n\$ _TX_END_INT](#). (R/WTC/SS)

RMT_CH n _ERR_INT_RAW (n : 0-3) The raw interrupt status of [RMT_CH \$n\$ _ERR_INT](#). (R/WTC/SS)

RMT_CH n _TX_THR_EVENT_INT_RAW (n : 0-3) The raw interrupt status of [RMT_CH \$n\$ _TX_THR_EVENT_INT](#). (R/WTC/SS)

RMT_CH n _TX_LOOP_INT_RAW (n : 0-3) The raw interrupt status of [RMT_CH \$n\$ _TX_LOOP_INT](#). (R/WTC/SS)

RMT_CH m _RX_END_INT_RAW (m : 4-7) The raw interrupt status of [RMT_CH \$m\$ _RX_END_INT](#). (R/WTC/SS)

RMT_CH m _ERR_INT_RAW (m : 4-7) The raw interrupt status of [RMT_CH \$m\$ _ERR_INT](#). (R/WTC/SS)

RMT_CH m _RX_THR_EVENT_INT_RAW (m : 4-7) The raw interrupt status of [RMT_CH \$m\$ _RX_THR_EVENT_INT](#). (R/WTC/SS)

RMT_CH n _DMA_ACCESS_FAIL_INT_RAW (n : 3) The raw interrupt status of [RMT_CH \$n\$ _DMA_ACCESS_FAIL_INT](#). (R/WTC/SS)

RMT_CH m _DMA_ACCESS_FAIL_INT_RAW (m : 7) The raw interrupt status of [RMT_CH \$m\$ _DMA_ACCESS_FAIL_INT](#). (R/WTC/SS)

Register 57.12. RMT_INT_ST_REG (0x0074)

(reserved) RMT_CH7_DMA_ACCESS_FAIL_INT_ST RMT_CH3_DMA_ACCESS_FAIL_INT_ST RMT_CH7_RX_THR_EVENT_INT_ST RMT_CH6_RX_THR_EVENT_INT_ST RMT_CH5_RX_THR_EVENT_INT_ST RMT_CH4_RX_THR_EVENT_INT_ST RMT_CH7_ERR_INT_ST RMT_CH6_ERR_INT_ST RMT_CH5_ERR_INT_ST RMT_CH4_ERR_INT_ST RMT_CH7_RX_END_INT_ST RMT_CH6_RX_END_INT_ST RMT_CH5_RX_END_INT_ST RMT_CH4_RX_END_INT_ST RMT_CH3_TX_LOOP_INT_ST RMT_CH2_TX_LOOP_INT_ST RMT_CH1_TX_LOOP_INT_ST RMT_CH0_TX_LOOP_INT_ST RMT_CH3_TX_THR_EVENT_INT_ST RMT_CH2_TX_THR_EVENT_INT_ST RMT_CH1_TX_THR_EVENT_INT_ST RMT_CH0_TX_THR_EVENT_INT_ST RMT_CH3_ERR_INT_ST RMT_CH2_ERR_INT_ST RMT_CH1_ERR_INT_ST RMT_CH0_ERR_INT_ST RMT_CH3_TX_END_INT_ST RMT_CH2_TX_END_INT_ST RMT_CH1_TX_END_INT_ST RMT_CH0_TX_END_INT_ST																																	
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

RMT_CH n _TX_END_INT_ST (n : 0-3) The masked interrupt status of [RMT_CH \$n\$ _TX_END_INT](#). (RO)

RMT_CH n _ERR_INT_ST (n : 0-3) The masked interrupt status of [RMT_CH \$n\$ _ERR_INT](#). (RO)

RMT_CH n _TX_THR_EVENT_INT_ST (n : 0-3) The masked interrupt status of [RMT_CH \$n\$ _TX_THR_EVENT_INT](#). (RO)

RMT_CH n _TX_LOOP_INT_ST (n : 0-3) The masked interrupt status of [RMT_CH \$n\$ _TX_LOOP_INT](#). (RO)

RMT_CH m _RX_END_INT_ST (m : 4-7) The masked interrupt status of [RMT_CH \$m\$ _RX_END_INT](#). (RO)

RMT_CH m _ERR_INT_ST (m : 4-7) The masked interrupt status of [RMT_CH \$m\$ _ERR_INT](#). (RO)

RMT_CH m _RX_THR_EVENT_INT_ST (m : 4-7) The masked interrupt status of [RMT_CH \$m\$ _RX_THR_EVENT_INT](#). (RO)

RMT_CH n _DMA_ACCESS_FAIL_INT_ST (n : 3) The masked interrupt status of [RMT_CH \$n\$ _DMA_ACCESS_FAIL_INT](#). (RO)

RMT_CH m _DMA_ACCESS_FAIL_INT_ST (m : 7) The masked interrupt status of [RMT_CH \$m\$ _DMA_ACCESS_FAIL_INT](#). (RO)

Register 57.13. RMT_INT_ENA_REG (0x0078)

(reserved) RMT_CH7_DMA_ACCESS_FAIL_INT_ENA RMT_CH3_DMA_ACCESS_FAIL_INT_ENA RMT_CH7_RX_THR_EVENT_INT_ENA RMT_CH6_RX_THR_EVENT_INT_ENA RMT_CH5_RX_THR_EVENT_INT_ENA RMT_CH4_RX_THR_EVENT_INT_ENA RMT_CH7_ERR_INT_ENA RMT_CH6_ERR_INT_ENA RMT_CH5_ERR_INT_ENA RMT_CH4_ERR_INT_ENA RMT_CH7_RX_END_INT_ENA RMT_CH6_RX_END_INT_ENA RMT_CH5_RX_END_INT_ENA RMT_CH4_RX_END_INT_ENA RMT_CH3_TX_LOOP_INT_ENA RMT_CH2_TX_LOOP_INT_ENA RMT_CH1_TX_LOOP_INT_ENA RMT_CH0_TX_LOOP_INT_ENA RMT_CH3_TX_THR_EVENT_INT_ENA RMT_CH2_TX_THR_EVENT_INT_ENA RMT_CH1_TX_THR_EVENT_INT_ENA RMT_CH0_TX_THR_EVENT_INT_ENA RMT_CH3_ERR_INT_ENA RMT_CH2_ERR_INT_ENA RMT_CH1_ERR_INT_ENA RMT_CH0_ERR_INT_ENA RMT_CH3_TX_END_INT_ENA RMT_CH2_TX_END_INT_ENA RMT_CH1_TX_END_INT_ENA RMT_CH0_TX_END_INT_ENA																																
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

RMT_CH n _TX_END_INT_ENA (n : 0-3) Write 1 to enable [RMT_CH \$n\$ _TX_END_INT](#). (R/W)

RMT_CH n _ERR_INT_ENA (n : 0-3) Write 1 to enable [RMT_CH \$n\$ _ERR_INT](#). (R/W)

RMT_CH n _TX_THR_EVENT_INT_ENA (n : 0-3) Write 1 to enable [RMT_CH \$n\$ _TX_THR_EVENT_INT](#). (R/W)

RMT_CH n _TX_LOOP_INT_ENA (n : 0-3) Write 1 to enable [RMT_CH \$n\$ _TX_LOOP_INT](#). (R/W)

RMT_CH m _RX_END_INT_ENA (m : 4-7) Write 1 to enable [RMT_CH \$m\$ _RX_END_INT](#). (R/W)

RMT_CH m _ERR_INT_ENA (m : 4-7) Write 1 to enable [RMT_CH \$m\$ _ERR_INT](#). (R/W)

RMT_CH m _RX_THR_EVENT_INT_ENA (m : 4-7) Write 1 to enable [RMT_CH \$m\$ _RX_THR_EVENT_INT](#). (R/W)

RMT_CH n _DMA_ACCESS_FAIL_INT_ENA (n : 3) Write 1 to enable [RMT_CH \$n\$ _DMA_ACCESS_FAIL_INT](#). (R/W)

RMT_CH m _DMA_ACCESS_FAIL_INT_ENA (m : 7) Write 1 to enable [RMT_CH \$m\$ _DMA_ACCESS_FAIL_INT](#). (R/W)

Register 57.14. RMT_INT_CLR_REG (0x007C)

(reserved)																																RMT_CH7_DMA_ACCESS_FAIL_INT_CLR																																RMT_CH3_DMA_ACCESS_FAIL_INT_CLR																																RMT_CH7_RX_THR_EVENT_INT_CLR																																RMT_CH6_RX_THR_EVENT_INT_CLR																																RMT_CH5_RX_THR_EVENT_INT_CLR																																RMT_CH4_RX_THR_EVENT_INT_CLR																																RMT_CH7_ERR_INT_CLR																																RMT_CH6_ERR_INT_CLR																																RMT_CH5_ERR_INT_CLR																																RMT_CH4_ERR_INT_CLR																																RMT_CH7_RX_END_INT_CLR																																RMT_CH6_RX_END_INT_CLR																																RMT_CH5_RX_END_INT_CLR																																RMT_CH4_RX_END_INT_CLR																																RMT_CH3_TX_LOOP_INT_CLR																																RMT_CH2_TX_LOOP_INT_CLR																																RMT_CH1_TX_LOOP_INT_CLR																																RMT_CH0_TX_LOOP_INT_CLR																																RMT_CH3_TX_THR_EVENT_INT_CLR																																RMT_CH2_TX_THR_EVENT_INT_CLR																																RMT_CH1_TX_THR_EVENT_INT_CLR																																RMT_CH0_TX_THR_EVENT_INT_CLR																																RMT_CH3_ERR_INT_CLR																																RMT_CH2_ERR_INT_CLR																																RMT_CH1_ERR_INT_CLR																																RMT_CH0_ERR_INT_CLR																																RMT_CH3_TX_END_INT_CLR																																RMT_CH2_TX_END_INT_CLR																																RMT_CH1_TX_END_INT_CLR																																RMT_CH0_TX_END_INT_CLR																																																																																									
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Reset																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																									
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

RMT_CH n _TX_END_INT_CLR (n : 0-3) Write 1 to clear **RMT_CH n _TX_END_INT**. (WT)

RMT_CH n _ERR_INT_CLR (n : 0-3) Write 1 to clear **RMT_CH n _ERR_INT**. (WT)

RMT_CH n _TX_THR_EVENT_INT_CLR (n : 0-3) Write 1 to clear **RMT_CH n _TX_THR_EVENT_INT**. (WT)

RMT_CH n _TX_LOOP_INT_CLR (n : 0-3) Write 1 to clear **RMT_CH n _TX_LOOP_INT**. (WT)

RMT_CH m _RX_END_INT_CLR (m : 4-7) Write 1 to clear **RMT_CH m _RX_END_INT**. (WT)

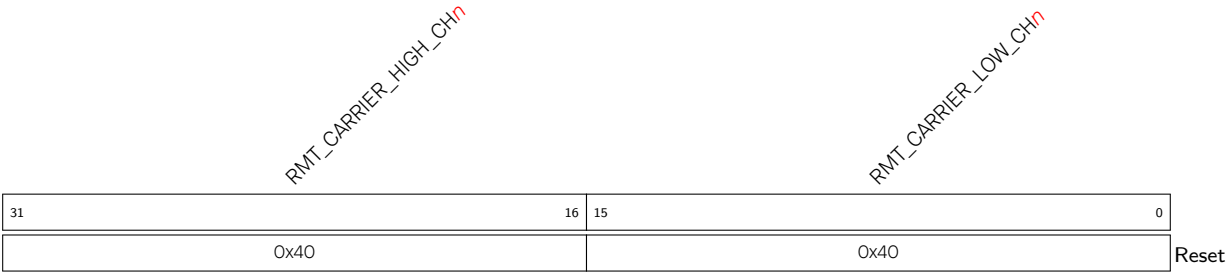
RMT_CH m _ERR_INT_CLR (m : 4-7) Write 1 to clear **RMT_CH m _ERR_INT**. (WT)

RMT_CH m _RX_THR_EVENT_INT_CLR (m : 4-7) Write 1 to clear **RMT_CH m _RX_THR_EVENT_INT**. (WT)

RMT_CH n _DMA_ACCESS_FAIL_INT_CLR (n : 3) Write 1 to clear **RMT_CH n _DMA_ACCESS_FAIL_INT**. (WT)

RMT_CH m _DMA_ACCESS_FAIL_INT_CLR (m : 7) Write 1 to clear **RMT_CH m _DMA_ACCESS_FAIL_INT**. (WT)

Register 57.15. RMT_CHnCARRIER_DUTY_REG (n: 0-3) (0x0080+0x4*n)



RMT_CARRIER_LOW_CHn Configures carrier wave's low level clock period for channel n.
Measurement unit: rmt_sclk
(R/W)

RMT_CARRIER_HIGH_CHn Configures carrier wave's high level clock period for channel n.
Measurement unit: rmt_sclk
(R/W)

Register 57.16. RMT_CH n _TX_LIM_REG (n : 0-3) (0x00A0+0x4* n)

(reserved)										RMT_LOOP_STOP_EN_CH ⁿ RMT_LOOP_COUNT_RESET_CH ⁿ RMT_TX_LOOP_CNT_EN_CH ⁿ				RMT_TX_LOOP_NUM_CH ⁿ				RMT_TX_LIM_CH ⁿ								
31											22	21	20	19	18					9	8					0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				0x80				Reset			

Reset

RMT_TX_LIM_CH n Configures the maximum entries that channel n can send out. (R/W)

RMT_TX_LOOP_NUM_CH n Configures the maximum loop count when Continuous TX mode is valid.
(R/W)

RMT_TX_LOOP_CNT_EN_CH n Configures whether to enable loop count.
0: No effect
1: Enable
(R/W)

RMT_LOOP_COUNT_RESET_CH n Configures whether to reset the loop count when continuous TX mode is enabled.
0: No effect
1: Reset
(WT)

RMT_LOOP_STOP_EN_CH n Configures whether to enable the loop send stop function after the loop counter counts to loop number for channel n .
0: No effect
1: Enable
(R/W)

Register 57.17. RMT_TX_SIM_REG (0x00C4)

(reserved)																												RMT_TX_SIM_EN RMT_TX_SIM_CH3 RMT_TX_SIM_CH2 RMT_TX_SIM_CH1 RMT_TX_SIM_CH0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																							
31																												5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

RMT_TX_SIM_CH n (n : 0-3) Configures whether to enable channel n to start sending data synchronously with other enabled channels.

0: No effect

1: Enable

(R/W)

RMT_TX_SIM_EN Configures whether to enable multiple of channels to start sending data synchronously.

0: No effect

1: Enable

(R/W)

Register 57.18. RMT_CH m _RX_LIM_REG (m : 4-7) (0x00B0, 0x00B4, 0x00B8, 0x00BC)

(reserved)																RMT_CH _m _RX_LIM_REG																
31																0																
0 0																0x80																Reset

RMT_CH m _RX_LIM_REG Configures the maximum entries that channel m can receive. (R/W)

Register 57.19. RMT_DATE_REG (0x00CC)

(reserved)																RMT_DATE																			
31				28				27																				0							
0				0				0				0				0x2201111																Reset			

RMT_DATE Version control register. (R/W)